

Arreglos - Ejercicios II

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo



Arreglos - Ejercicios II

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

Objetivo General

Practicar el uso de arreglos para almacenar y manipular colecciones de datos de manera eficiente.

Objetivos Específicos

- Declarar elementos de un arreglo unidimensional, bidimensional y dinámicos para almacenar múltiples valores del mismo tipo.
- Realizar operaciones básicas con arreglos como sumas, ordenamientos y búsquedas.

Ejercicio I de búsqueda lineal

"Búsqueda lineal en un arreglo de letras"

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      char arreglo[10];    // Arreglo de máximo 10 letras (tipo char)
6      int tamano;
7      char letraBuscada;
8      bool encontrada = false; // Variable para saber si se encontró la letra
9
10     // Solicitar el tamaño del arreglo
11     cout << "¿Cuántas letras deseas ingresar? (máximo 10): ";
12     cin >> tamano;
13
14     // Validar que el tamaño sea entre 1 y 10
15     if (tamano < 1 || tamano > 10) {
16         cout << "Tamaño inválido. Debe ser entre 1 y 10." << endl;
17         return 0;
18     }
19
20     // Llenar el arreglo con letras
21     cout << "\n--- Ingresando letras al arreglo ---\n";
22     for (int i = 0; i < tamano; i++) {
23         cout << "arreglo[" << i << "]: ";
24         cin >> arreglo[i];
25     }
26
27     // Mostrar el arreglo completo
28     cout << "\n--- Letras ingresadas ---\n";
29     for (int i = 0; i < tamano; i++) {
30         cout << "Posición [" << i << "]: " << arreglo[i] << endl;
31     }
32
33     // Solicitar la letra a buscar
```

```
33     // Solicitar la letra a buscar
34     cout << "\nIngrese la letra que desea buscar: ";
35     cin >> letraBuscada;
36
37     // Realizar la búsqueda lineal
38     cout << "\n--- Resultado de la búsqueda ---\n";
39     for (int i = 0; i < tamano; i++) {
40         if (arreglo[i] == letraBuscada) {
41             cout << "Letra '" << letraBuscada << "' encontrada en la posición [" << i << "].\n" << endl;
42             encontrada = true;
43         }
44     }
45
46     // Si no se encontró
47     if (!encontrada) {
48         cout << "Letra '" << letraBuscada << "' no se encuentra en el arreglo." << endl;
49     }
50
51     return 0;
52 }
```

Este programa permite practicar el concepto de **búsqueda lineal**, también llamada **búsqueda secuencial**.

1. Primero, el usuario define cuántas letras quiere ingresar (máximo 10).
2. Luego, llena el arreglo con letras, una por una.
3. El programa muestra el contenido del arreglo, para que el usuario pueda verificar.
4. Después, el usuario ingresa la **letra que desea buscar**.
5. El programa recorre el arreglo **de principio a fin** (índice 0 hasta n-1) comparando cada posición con la letra buscada.
6. Si la encuentra, muestra la **posición o posiciones** donde aparece.
7. Si no aparece en ninguna, muestra un mensaje indicando que **no se encontró**.

```
¿Cuántas letras deseas ingresar? (máximo 10): 5
--- Ingresando letras al arreglo ---
arreglo[0]: a
arreglo[1]: b
arreglo[2]: c
arreglo[3]: d
arreglo[4]: f
--- Letras ingresadas ---
Posición [0]: a
Posición [1]: b
Posición [2]: c
Posición [3]: d
Posición [4]: f
Ingrese la letra que desea buscar: d
--- Resultado de la búsqueda ---
Letra 'd' encontrada en la posición [3].
```


Ejercicio I de búsqueda binario

"Búsqueda binaria en un arreglo ordenado de números enteros"

main.cpp

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      // Arreglo ordenado de 10 números
5      int arreglo[] = {2, 5, 8, 12, 16, 23, 38, 45, 56, 67};
6      int inicio = 0;
7      int fin = 9;
8      int medio;
9      int numeroBuscado;
10     bool encontrado = false;
11     // Mostrar el arreglo
12     cout << "--- Arreglo ordenado ---\n";
13     for (int i = 0; i < 10; i++) {
14         cout << arreglo[i] << " ";
15     }
16     cout << endl;
17     // Solicitar el número a buscar
18     cout << "\nIngrese el número que desea buscar: ";
19     cin >> numeroBuscado;
20     // Búsqueda binaria
21     while (inicio <= fin) {
22         medio = (inicio + fin) / 2;
23         if (arreglo[medio] == numeroBuscado) {
24             cout << "Número encontrado en la posición [" << medio << "].\n" << endl;
25             encontrado = true;
26             break;
27         } else if (numeroBuscado < arreglo[medio]) {
28             fin = medio - 1; // Buscar en la mitad izquierda
29         } else {
30             inicio = medio + 1; // Buscar en la mitad derecha
31         }
32     }
33     if (!encontrado) {
34         cout << "El número no se encuentra en el arreglo.\n" << endl;
35     }
36     return 0;
37 }
```



Explicación del programa:

- Se trabaja con un arreglo ordenado de 10 números (int).
- Se muestra el arreglo en pantalla.
- El usuario ingresa el número que desea buscar.
- El programa aplica el algoritmo de búsqueda binaria, que consiste en:
 - Dividir el arreglo a la mitad.
 - Comparar el número buscado con el del medio.
 - Si es menor, buscar en la mitad izquierda.
 - Si es mayor, buscar en la mitad derecha.
- Esto se repite hasta encontrar el número o terminar las posibilidades.
- Finalmente, se muestra el resultado: posición donde se encuentra o mensaje de que no fue encontrado.

Output

```
--- Arreglo ordenado ---
2 5 8 12 16 23 38 45 56 67
```

```
Ingrese el número que desea buscar: 16
Número encontrado en la posición [4].
```

✓ Ventajas de la búsqueda binaria:

- Mucho más rápida que la búsqueda lineal, especialmente en arreglos grandes.
- Ideal para datos que ya están **ordenados**.

Ejercicio I de ordenamiento

"Ordenar números con la función sort() de C++

```
main.cpp
1 #include <iostream>
2 #include <algorithm> // Necesario para usar sort()
3 using namespace std;
4 int main() {
5     int arreglo[10]; // Arreglo de hasta 10 elementos
6     int tamano;
7     // Solicitar el tamaño del arreglo
8     cout << "¿Cuántos números deseas ingresar? (máximo 10): ";
9     cin >> tamano;
10    // Validar que el tamaño esté dentro del rango permitido
11    if (tamano < 1 || tamano > 10) {
12        cout << "Tamaño inválido. Debe ser entre 1 y 10." << endl;
13        return 0;
14    }
15    // Ingreso de datos al arreglo
16    cout << "\n--- Ingrese los números ---\n";
17    for (int i = 0; i < tamano; i++) {
18        cout << "Número [" << i << "]: ";
19        cin >> arreglo[i];
20    }
21    // Mostrar el arreglo antes de ordenar
22    cout << "\nArreglo antes de ordenar:\n";
23    for (int i = 0; i < tamano; i++) {
24        cout << arreglo[i] << " ";
25    }
26    cout << endl;
27    // Ordenar el arreglo usando sort()
28    sort(arreglo, arreglo + tamano);
29    // Mostrar el arreglo ordenado
30    cout << "\nArreglo ordenado de menor a mayor:\n";
31    for (int i = 0; i < tamano; i++) {
32        cout << arreglo[i] << " ";
33    }
34    cout << endl;
35    return 0;
36 }
```

Output

```
¿Cuántos números deseas ingresar? (máximo 10): 3

--- Ingrese los números ---
Número [0]: 2
Número [1]: -10
Número [2]: 5

Arreglo antes de ordenar:
2 -10 5

Arreglo ordenado de menor a mayor:
-10 2 5
```

🧠 Explicación del programa:

- 1.El usuario indica **cuántos números** desea ingresar (máximo 10).
- 2.Se llena el arreglo con esos números usando un for.
- 3.Se muestra el **arreglo antes de ordenarlo**.
- 4.Se llama a la función sort(arreglo, arreglo + tamano) para ordenarlo de menor a mayor.
- 5.Finalmente, se muestra el **arreglo ya ordenado**.

🔧 ¿Qué hace sort()?

La función sort() pertenece a la librería <algorithm> y permite **ordenar un rango de elementos** con solo una línea de código. Es muy rápida y confiable.

Arreglos - Ejercicios II

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

Arreglos - Ejercicios II

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

